

基于 C 语言的轻量化 DBMS 原型实验 指导书

DBMS_C 教学实践材料

2026 年 5 月 24 日

使用说明

本指导书面向数据库课程实践、计算机专业低年级学生以及 DBMS_C 项目阅读者。它围绕当前仓库中的真实 C 源码、测试入口、Trace 输出和图表组织实验内容，帮助读者逐步理解一个轻量化教学型 DBMS 原型的内部机制。

本指导书不是毕业论文，也不是工业级数据库手册。后续章节将遵循“先补充或确认真实 CTest，再撰写章节正文”的方式逐章完善；本轮仅完整整理第 0 章，并为第 1 章到第 12 章保留统一占位结构。

目录

第 0 章	引言与环境准备	1
0.1	编写目的	1
0.2	项目定位	1
0.3	学习主线	1
0.4	项目目录概览	1
0.5	环境准备	3
0.6	构建项目	3
0.7	运行测试与测试驱动的实验组织方式	4
0.8	运行系统	5
0.9	Trace 模式说明	6
0.10	如何阅读本指导书	6
0.11	本章小结	6
第 1 章	基础组件与工具库实验	7
1.1	实验目的	7
1.2	模块作用	7
1.3	相关源码文件	7
1.4	核心组件分类	7
1.5	在 DBMS 主链路中的位置	7
1.6	配套测试	9
1.7	关键代码讲解	10
1.8	运行与观察	12
1.9	思考题	12
第 2 章	文件与页式存储实验	13
2.1	实验目的	13
2.2	模块作用	13
2.3	相关源码文件	13
2.4	核心对象职责	14
2.5	文件块与内存页的关系	14
2.6	配套测试	14
2.7	关键代码讲解	15
2.8	运行与观察	18
2.9	思考题	19

第 3 章 日志管理实验	20
3.1 实验目的	20
3.2 模块作用	20
3.3 相关源码文件	20
3.4 核心对象职责	21
3.5 日志页与日志记录组织	21
3.6 配套测试	22
3.7 关键代码讲解	22
3.8 运行与观察	25
3.9 思考题	26
第 4 章 缓冲区管理实验	27
4.1 实验目的	27
4.2 模块作用	27
4.3 相关源码文件	27
4.4 核心对象关系	27
4.5 pin/unpin 机制	27
4.6 modified、dirty 与 flush 机制	29
4.7 配套测试	30
4.8 关键代码讲解	30
4.9 运行与观察	32
4.10 思考题	32
第 5 章 事务、并发与恢复实验	34
5.1 实验目的	34
5.2 相关源码文件	34
5.3 配套测试	34
5.4 实验内容	34
5.5 思考题	34
第 6 章 记录管理实验	35
6.1 实验目的	35
6.2 相关源码文件	35
6.3 配套测试	35
6.4 实验内容	35
6.5 思考题	35
第 7 章 元数据管理实验	36
7.1 实验目的	36
7.2 相关源码文件	36

7.3	配套测试	36
7.4	实验内容	36
7.5	思考题	36
第 8 章	SQL 解析实验	37
8.1	实验目的	37
8.2	相关源码文件	37
8.3	配套测试	37
8.4	实验内容	37
8.5	思考题	37
第 9 章	查询计划与扫描执行实验	38
9.1	实验目的	38
9.2	相关源码文件	38
9.3	配套测试	38
9.4	实验内容	38
9.5	思考题	38
第 10 章	更新执行实验	39
10.1	实验目的	39
10.2	相关源码文件	39
10.3	配套测试	39
10.4	实验内容	39
10.5	思考题	39
第 11 章	SQL 执行过程 Trace 可视化实验	40
11.1	实验目的	40
11.2	相关源码文件	40
11.3	配套测试	40
11.4	实验内容	40
11.5	思考题	40
第 12 章	综合实验	41
12.1	实验目的	41
12.2	相关源码文件	41
12.3	配套测试	41
12.4	实验内容	41
12.5	思考题	41

第 0 章 引言与环境准备

本章用于建立阅读 DBMS_C 的共同起点。读者在进入具体模块实验之前，需要先知道本项目的教学定位、源码目录如何组织、如何稳定构建和测试，以及后续章节为什么要采用“先测试、再写实验内容”的组织方式。

0.1 编写目的

本指导书面向计算机专业低年级学生、数据库课程实践和 DBMS_C 项目阅读者。它的目标不是让读者一次性掌握所有数据库实现细节，而是通过“源码阅读 + 测试运行 + Trace 观察 + 图表理解”的方式，逐步理解一个轻量化 DBMS 原型从 SQL 输入到文件存储访问的大致过程。

后续各章会围绕真实源码展开：先确认或补充可运行的测试，再把测试目标、运行命令、预期现象、观察点和对应源码位置写入章节正文。这样做的好处是，实验内容不只停留在文字说明，而是可以被当前工程实际验证。

0.2 项目定位

DBMS_C 是一个基于 C 语言实现的轻量化教学型 DBMS 原型，已经包含 SQL 解析、查询计划、扫描执行、记录管理、元数据管理、事务日志、缓冲区管理、文件页式存储等模块。它强调“可运行、可观察、可阅读”，适合作为数据库系统课程中的实验材料和源码阅读对象。

需要注意的是，本项目不是完整工业级数据库系统。它不会覆盖工业数据库中的全部 SQL 标准、优化器规则、并发事务隔离级别、存储引擎能力和高性能工程化细节。本指导书也不是论文式说明文档，而是配套 DBMS_C 源码的实验指导书。

0.3 学习主线

阅读 DBMS_C 时，可以沿着从用户输入向底层存储逐步下沉的方向理解系统。图 0-1 不是完整函数调用图，而是一张学习路线图：上层更接近 SQL 和用户观察结果，下层更接近数据页、文件和事务支撑。

图 0-1 中，各层的作用可以这样理解：SQL 输入层负责接收用户命令；SQL 解析层把文本转换为内部数据对象；查询计划层决定如何组合表扫描、选择和投影；Scan 扫描执行层以统一接口向上提供记录；记录访问层负责表记录的定位与读写；元数据层保存表结构、视图和索引相关信息；事务与日志层为更新和恢复提供支撑；缓冲区层管理内存中的页；文件与页式存储层最终负责磁盘文件和页面读写。Trace 模式不改变执行语义，只帮助读者观察这些阶段的大致过程。

0.4 项目目录概览

DBMS_C 的目录不是按论文章节划分，而是按数据库执行链路和模块职责组织。表 0-1 根据当前仓库中的真实目录整理，便于读者在后续实验中快速定位源码。

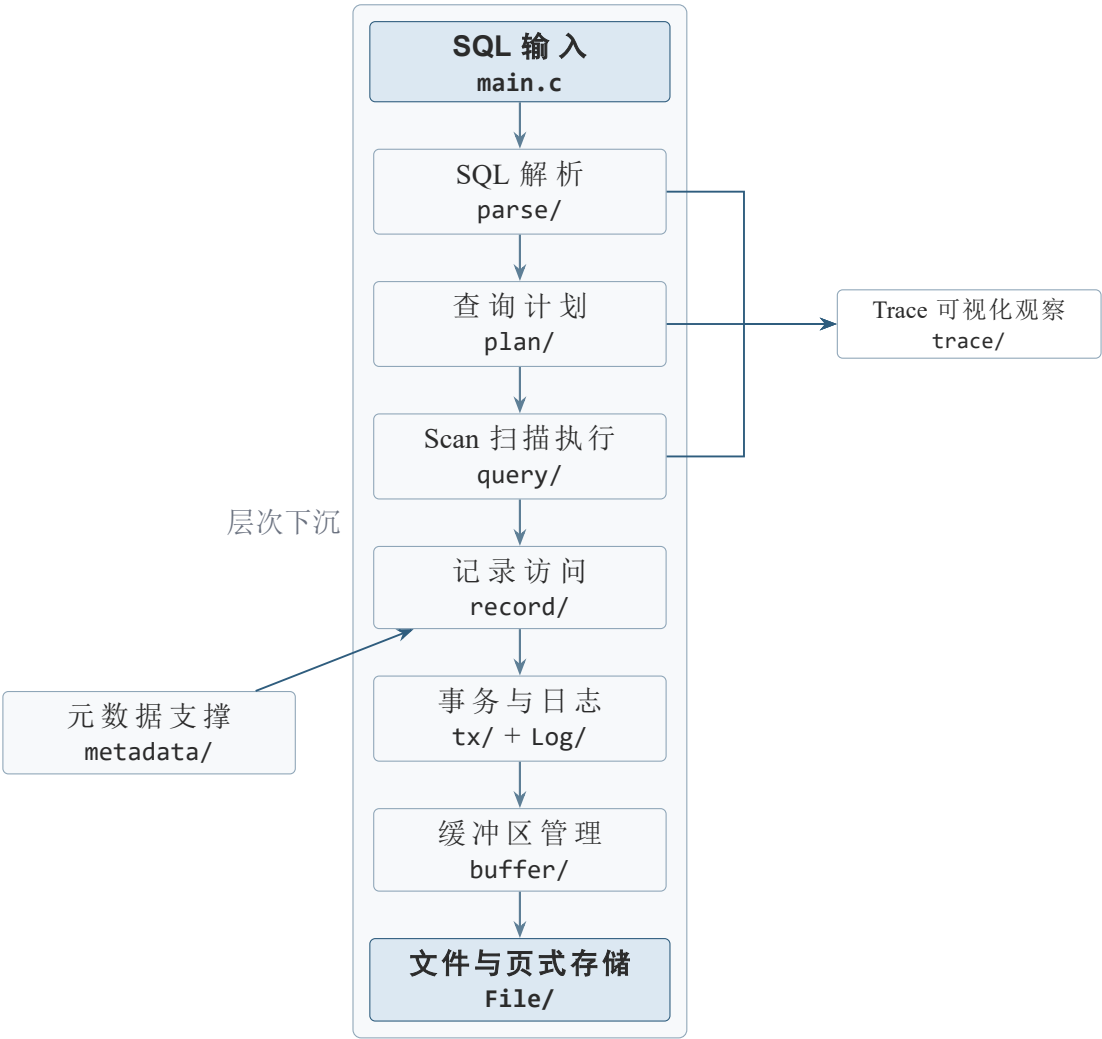


图 0-1 DBMS_C 学习主线图

表 0-1 DBMS_C 项目目录职责表

目录或文件	主要职责
Lib/	基础工具库，包括字符串、字节缓冲、列表、映射、向量、分词辅助等通用组件。
File/	文件块、页对象和文件管理，是理解页式存储的入口。
Log/	日志管理相关实现，为事务提交、回滚和恢复提供基础。
buffer/	缓冲区管理和 LRU 替换策略，负责内存页与磁盘页之间的协调。
tx/	事务、并发控制、恢复管理以及事务持有的缓冲区列表。
record/	Schema、Layout、RID、RecordPage、TableScan 等记录层结构。
metadata/	表、字段、视图、索引和统计信息等元数据管理。
parse/	Lexer、Parser 以及各类 SQL 命令的数据对象。
plan/	查询计划和更新计划的构造入口。
query/	Scan 接口、表达式、谓词、选择、投影和笛卡尔积扫描等执行层对象。
index/	当前仓库中的哈希索引相关实现。
DBMS/	SimpleDB 初始化入口，连接文件、日志、缓冲区、元数据和规划器。
main.c	CLI 入口，负责读取命令、分发 SQL、执行事务命令和输出结果。
test/	基于 CMocka 的已有测试文件。
docs/	说明文档、Trace 文档和本实验指导书。

推荐阅读顺序是: Lib/ → File/ → Log/ → buffer/ → tx/ → record/ → metadata/ → parse/ → plan/ 与 query/ → 更新执行 → Trace → 综合实验。这个顺序先建立底层对象概念，再理解上层 SQL 如何使用这些能力。

0.5 环境准备

本指导书默认在 Windows 环境下进行实验。当前项目的稳定构建方式依赖 CMake、MinGW 编译器和 mingw32-make。编译本指导书本身还需要 XeLaTeX。

建议准备以下环境：

- Windows 操作系统；
- MinGW 或等价的 GCC 工具链，并确保 mingw32-make 可用；
- CMake；
- XeLaTeX，用于编译本指导书；
- 可选编辑器：VS Code、CLion、Trae 或 Codex。

如果不同机器上的工具链路径不同，应优先保证命令行中能直接执行 cmake、mingw32-make 和 xelatex。后续章节不会假定某一个 IDE 的专有功能。

0.6 构建项目

当前 Windows/Codex 环境下，DBMS_C 已验证的稳定构建命令如表 0-2 所示。请先进入项目根目录，再进行配置和编译。

表 0-2 构建与测试命令表

命令	作用
cd D:\code\DBMS\NewDBMS\DBMS_C	进入 DBMS_C 项目目录。
cmake -S . -B build	根据当前 CMakeLists.txt 配置构建目录。
mingw32-make -C build -j8 SHELL=cmd.exe	使用 mingw32-make 编译项目，并显式覆盖 Makefile 中的 shell。
ctest --test-dir build --output-on-failure	运行已注册测试，并在失败时输出详细信息。
bin\NewDBMS.exe	启动交互式 DBMS 程序。

代码 0-1 稳定构建命令

```
cd D:\code\DBMS\NewDBMS\DBMS_C
cmake -S . -B build
mingw32-make -C build -j8 SHELL=cmd.exe
```

这里使用 mingw32-make -C build -j8 SHELL=cmd.exe，是因为当前构建目录在 Windows 下使用 Makefile 时，默认 shell 可能指向 /bin/sh。在 Codex/Trae 这类环境中，调用 Git/MSYS 的 shell 曾经触发过构建阶段问题。因此，本指导书把 mingw32-make 并覆盖 SHELL=cmd.exe 作为当前稳定主命令，不把 cmake --build build 作为默认主命令。

0.7 运行测试与测试驱动的实验组织方式

项目测试通过 CTest 统一运行：

代码 0-2 运行已有测试

```
ctest --test-dir build --output-on-failure
```

ctest 用于确认已有模块测试是否通过。后续每章新增或调整测试后，都要重新运行完整 CTest，确保没有破坏已有测试。如果新增测试，测试数量可能增加；如果测试失败，应优先查看第一条真实失败信息，而不是同时修改多个无关位置。

从第 1 章开始，本指导书采用测试驱动的实验组织方式：

- 1. 每个模块实验在正式撰写章节前，先补充或确认一个真实可运行的测试。
- 2. 测试必须基于当前仓库真实 API，不能编造不存在的函数、结构体或路径。
- 3. 测试通过后，章节中再写入测试文件路径、测试目标、运行命令、预期结果、观察点和对应源码位置。
- 4. 文档中不能写不存在或没有跑通的测试。

- 5. 如果某章暂时没有测试，就只能写“后续补充”，不能伪造测试结果。
- 6. 每新增一个测试后都要运行完整 CTest。
- 7. 第 0 章本身不新增模块测试，只记录当前工程的基线构建与测试方法。

表 0-3 给出后续章节与测试策略的对应关系。当前状态均为 TODO，表示需要在后续轮次中先处理测试，再完善章节正文。

表 0-3 后续章节与测试策略对照表

章节	测试策略
第 1 章基础组件与工具库实验	TODO：先补充或确认 Lib 相关 CTest，再撰写章节内容。
第 2 章文件与页式存储实验	TODO：先补充或确认 File/Page 相关 CTest，再撰写章节内容。
第 3 章日志管理实验	TODO：先补充或确认 Log 相关 CTest，再撰写章节内容。
第 4 章缓冲区管理实验	TODO：先补充或确认 buffer 相关 CTest，再撰写章节内容。
第 5 章事务、并发与恢复实验	TODO：先补充或确认 tx/recovery 相关 CTest，再撰写章节内容。
第 6 章记录管理实验	TODO：先补充或确认 record 相关 CTest，再撰写章节内容。
第 7 章元数据管理实验	TODO：先补充或确认 metadata 相关 CTest，再撰写章节内容。
第 8 章 SQL 解析实验	TODO：先补充或确认 parse 相关 CTest，再撰写章节内容。
第 9 章查询计划与扫描执行实验	TODO：先补充或确认 plan/query 相关 CTest，再撰写章节内容。
第 10 章更新执行实验	TODO：先补充或确认更新执行相关 CTest，再撰写章节内容。
第 11 章 SQL 执行过程 Trace 可视化实验	TODO：先确认 Trace 行为和可观察示例，再撰写章节内容。
第 12 章综合实验	TODO：在前序实验稳定后设计综合测试和综合任务。

0.8 运行系统

构建完成后,主程序位于项目根目录下的 bin/NewDBMS.exe。在 Windows PowerShell 中可以这样启动：

代码 0-3 启动 DBMS_C

```
cd D:\code\DBMS\NewDBMS\DBMS_C
bin\NewDBMS.exe
```

程序启动后会进入交互式 SQL 提示符。读者可以输入 SQL 语句并以分号结束，也可以输入 commit、rollback、trace on、trace off、exit 等元命令。输入 exit 会退

出系统；按当前程序逻辑，退出阶段会提交当前事务。因此，如果 Trace 已开启，读者可能在关闭数据库时看到一段退出阶段的 COMMIT Trace。

0.9 Trace 模式说明

Trace 模式用于观察 SQL 在 DBMS_C 内部的大致执行过程，包括解析、计划、扫描、更新、事务和输出等阶段。它的定位是 CLI 教学可视化，不是 GUI，也不改变普通 SQL 执行语义。

当前仓库中已经有 docs/trace_mode.md，用于记录 Trace 模式的开启方式、示例 SQL 和当前限制。本指导书第 11 章会进一步围绕 Trace 设计实验；第 0 章只要求读者知道 Trace 是一种观察执行链路的辅助工具。

0.10 如何阅读本指导书

建议按以下方式使用本指导书：

1. 先阅读每章实验目的，明确本章要理解的问题。
2. 再查看相关源码文件，确认结构体、函数和目录都来自真实仓库。
3. 再运行本章配套测试，观察测试是否通过以及输出是否符合预期。
4. 对照图、表、代码和 Trace 输出理解执行链路。
5. 最后完成思考题，把模块放回 DBMS 的整体执行路径中理解。

如果某章暂时只有 TODO，占位内容表示该章还没有完成“先测试、再写章节”的准备工作。不要把 TODO 当成已完成实验，也不要根据 TODO 推断源码中已经存在某个未验证接口。

0.11 本章小结

本章完成了指导书的起步准备：明确了 DBMS_C 的教学型原型定位，给出了从 SQL 输入向底层存储下沉的学习主线，整理了真实项目目录职责，说明了 Windows 环境下稳定构建、测试和运行系统的方法，并规定了后续章节采用测试驱动的实验组织方式。

从下一章开始，每一章都应先围绕对应模块补充或确认真实可运行的 CTest，再把测试入口、源码位置和观察结果写入章节正文。下一步建议从第 1 章基础组件与工具库实验开始。

第 1 章 基础组件与工具库实验

1.1 实验目的

本章用于理解 DBMS_C 为什么需要自定义基础组件。C 语言本身没有内建字符串对象、动态数组、链表、映射容器和统一字节缓冲区；如果每个上层模块都直接操作裸指针和裸数组，SQL 解析、记录布局、页式存储和元数据管理会很快变得难以阅读和验证。

本章实验的目标是先确认 Lib/ 中若干基础组件可以独立工作，再观察它们如何支撑上层 DBMS 模块。实验范围以当前真实 API 和已经跑通的 test/Lib/LibBasicTest.c 为准，不扩展未验证接口。

1.2 模块作用

Lib/ 为上层模块提供字符串、动态容器、字节缓冲和词法切分等基础能力。它不是一个独立数据库模块，而是许多数据库对象共同依赖的工具层：表名、字段名和文件名依赖字符串组件；字段列表、表列表和谓词项依赖容器组件；页内数据编码依赖字节缓冲组件；SQL 解析入口依赖词法切分组件。

这些基础组件越稳定，上层模块越容易被单独测试。反过来说，如果基础组件存在内存泄漏、越界读写或比较规则错误，问题可能会在 Parser、Record、Metadata、Buffer 等更高层才表现出来，定位会更加困难。

1.3 相关源码文件

表 1-1 列出了本章展开的真实文件。这里的“展开”表示本章会描述其基本职责，并在配套测试中至少覆盖其中一部分稳定 API。

本章不展开 rlock.h、BlockLockManager.c/.h、RBT 和 Trie。其中，RBT 作为 CMap 的内部依赖出现，暂不作为独立实验目标；Trie 和其他工具文件后续只有在确认主链路用途和测试入口后再展开。

1.4 核心组件分类

从实验角度看，Lib/ 可以先按用途分为五类。表 1-2 给出了本章的分类方式和对对应观察点。

这种分类不是源码目录的全部内容，而是本章实验的最小切入点。先确认这些组件独立可用，再阅读上层模块会更顺畅。

1.5 在 DBMS 主链路中的位置

图 1-1 展示了 Lib/ 对上层模块的支撑关系。图中不是函数调用顺序，而是“基础能力被谁使用”的关系。

CString 支撑 SQL 文本、字段名、表名、文件名等字符串对象；CVector、CList

表 1-1 本章涉及的 Lib 源码文件

文件	本章关注点
Lib/CString.h, Lib/CString.c	字符串创建、追加、获取内容、长度和比较。
Lib/CVector.h, Lib/CVector.c	按元素大小管理动态数组，支持插入、读取、查找和设置。
Lib/CList.h, Lib/CList.c	单链表容器，支持追加、按下标读取、contains、浅拷贝和深拷贝等行为。
Lib/CMap.h, Lib/CMap.c	基于红黑树包装的映射容器，本章只验证最小插入、查找和更新。
Lib/map.h	元数据等模块使用的宏式 map，本章只作为目录职责说明，不新增单独测试。
Lib/ByteBuffer.h, Lib/ByteBuffer.c	连续字节缓冲区，支持 int、long、bytes 等读写。
Lib/StreamTokenizer.h, Lib/StreamTokenizer.c	SQL 文本的轻量 token 切分工具。
Lib/Error.h, Lib/Error.c	旧错误对象，当前已标记为 legacy/deprecated，本章只验证默认初始化。

表 1-2 Lib 组件分类表

分类	代表文件	观察点
字符串组件	CString.h/.c	是否能保存、追加和比较 SQL 文本中的名称。
容器组件	CVector.h/.c, CList.h/.c, CMap.h/.c	是否能保存字段列表、表列表、谓词项和键值映射。
字节缓冲组件	ByteBuffer.h/.c	是否能按小端方式把整数编码到连续字节区并读回。
词法切分组件	StreamTokenizer.h/.c	是否能把 SQL 文本分成 WORD、NUMBER、STRING、DELIM 等 token。
错误处理辅助组件	Error.h/.c	旧错误结构体是否能默认初始化；新代码优先使用 error/DBError.h。

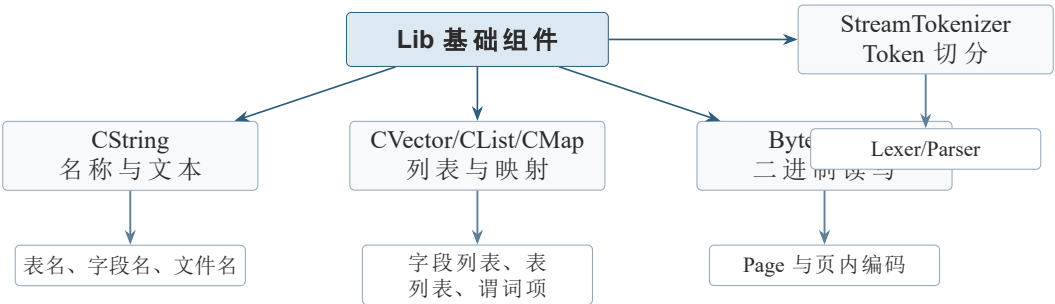


图 1-1 Lib 支撑 DBMS 上层模块关系图

和 CMap 支撑字段列表、表列表、谓词项、元数据组织等结构；ByteBuffer 支撑页内字节读写和底层数据编码；StreamTokenizer 支撑 SQL 解析前的词法切分。

图 1-2 用一个简单的整数写入示意 ByteBuffer 的基本思想：写入整数时，缓冲区从当前 position 开始连续写入 4 个字节；bufferFlip 会把 limit 设为已经写入的位置，并把 position 归零，随后即可按相同布局读回。

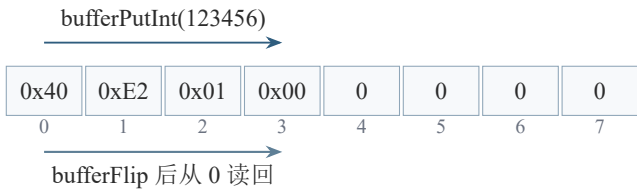


图 1-2 ByteBuffer 字节布局示意图

图 1-3 展示了 StreamTokenizer 对简单 SQL 片段的切分结果。它只负责把字符流分段，并记录 token 类型、起始位置、长度和整数值；更高层的语法含义仍由 Lexer/Parser 处理。



图 1-3 StreamTokenizer token 切分示意图

1.6 配套测试

本章新增并跑通的配套测试文件是：

- test/Lib/LibBasicTest.c
- 测试名称为 LibBasicTest。它验证了以下最小行为：
- CString：创建、追加、获取内容、长度和比较。
- CVector：插入整数、读取元素、查询大小、查找和设置。
- CList：追加、按下标读取、contains。
- CMap：插入、查找、通过 CMapPut 更新。
- ByteBuffer：写入 int、flip 后读回。
- StreamTokenizer：对简单 SQL 文本切分 WORD、DELIM、NUMBER。

- **Error:** 旧错误对象默认初始化。

运行本章测试可以使用 CTest 的正则过滤：

代码 1-1 运行 Lib 相关测试

```
ctest --test-dir build --output-on-failure -R Lib
```

完整回归测试仍然使用：

代码 1-2 运行完整测试

```
ctest --test-dir build --output-on-failure
```

本轮新增测试后，完整 CTest 已从 13 个测试增加到 14 个测试，并通过 14/14。

1.7 关键代码讲解

第一段代码来自 Lib/CString.c 的追加逻辑。它先计算追加后的长度，再确保容量足够，最后把新内容接到已有字符串尾部。

代码 1-3 CStringAppendCStr 的核心逻辑

```
1 void CStringAppendCStr(CString* cs, const char* str) {  
2     if (!cs || !str) return;  
3  
4     size_t appendLen = strlen(str);  
5     size_t newLen = cs->length + appendLen;  
6  
7     if (!ensureCapacity(cs, newLen + 1)) return;  
8  
9     strcat(cs->data + cs->length, str);  
10    cs->length = newLen;  
11 }
```

这段代码说明 CString 不只是保存一个 char*，还维护当前长度和容量。上层模块使用 CStringGetPtr 读取内容时，不需要关心扩容细节。

第二段代码来自 Lib/CVector.c 的尾插逻辑。它根据元素大小计算目标地址，并根据是否提供 Copy 回调决定复制方式。

代码 1-4 CVectorPushBack 的核心逻辑

```
1 void CVectorPushBack(CVector *vec, const void *value) {
2     if (!vec || !value) {
3         return;
4     }
5
6     if (vec->size + 1 > vec->capacity && !CVectorGrow(vec, vec->size + 1)) {
7         return;
8     }
9
10    void *dest = (char *)vec->data + vec->size * vec->elem_size;
11    if (vec->Copy) {
12        vec->Copy(dest, value);
13    } else {
14        memcpy(dest, value, vec->elem_size);
15    }
16    vec->size++;
17 }
```

这段代码体现了 C 语言中泛型容器的常见做法：容器不知道元素类型，只知道元素大小和可选回调函数。

第三段代码来自 Lib/ByteBuffer.c。写入整数时，代码按字节写入并移动 position；读出时按同样的字节布局恢复整数。

代码 1-5 ByteBuffer 写入 int 的核心逻辑

```
1 ByteBufferError bufferPutInt(ByteBuffer* buffer, int32_t data){
2     if (!buffer || !buffer->data) return BYTEBUFFER_ERROR_NULL;
3     if (buffer->position + 4 > buffer->limit) return BYTEBUFFER_ERROR_BOUNDS;
4     buffer->data[buffer->position++] = data & 255;
5     buffer->data[buffer->position++] = (data >> 8) & 255;
6     buffer->data[buffer->position++] = (data >> 16) & 255;
7     buffer->data[buffer->position++] = (data >> 24) & 255;
8     return BYTEBUFFER_OK;
9 }
```

这段代码和 File/Page 模块关系很近。后续学习页式存储时，会看到页内整数和字符串最终都需要落到字节数组中。

第四段代码来自 Lib/StreamTokenizer.c。它识别由字母或下划线开头的 WORD token，并通过 start 和 length 描述 token 内容。

代码 1-6 StreamTokenizer 识别 WORD 的核心逻辑

```
1 if (isalpha((unsigned char)c) || c == '_') {  
2     while (isalnum((unsigned char)*st->pos) || *st->pos == '_') st->pos++;  
3     st->length = (int)(st->pos - st->start);  
4     return (st->type = STT_WORD);  
5 }
```

这说明 StreamTokenizer 只做轻量词法切分，不判断 select、from、where 的语法位置。语法规则会在更高层解析器中处理。

1.8 运行与观察

本章建议先运行 Lib 相关测试：

代码 1-7 观察 Lib 测试

```
ctest --test-dir build --output-on-failure -R Lib
```

观察时重点关注四件事：

- 基础组件是否能在不启动数据库主程序的情况下独立工作。
- 字符串和容器是否能够支撑字段名、表名、列表和映射等上层数据结构。
- ByteBuffer 是否能完成基本二进制写入、翻转和读回。
- StreamTokenizer 是否能把 SQL 文本切成可供解析器继续处理的 token。

如果本章测试失败，应先看 LibBasicTest 的第一条失败信息，再回到对应头文件和实现文件确认真实 API 行为。

1.9 思考题

1. 为什么 C 语言项目需要自己封装 CString 和 CVector？
2. ByteBuffer 与后续 File/Page 模块之间是什么关系？
3. StreamTokenizer 为什么可以放在 Lib/，而不是直接写进 parse/？
4. 如果基础组件存在内存泄漏，会影响哪些上层模块？请结合表名、字段列表、页数据和 SQL 解析举例说明。

第 2 章 文件与页式存储实验

2.1 实验目的

本章用于理解 DBMS_C 如何把上层记录访问落实到底层文件块和内存页。数据库表、日志和记录并不是直接以“对象”的形式写入磁盘，而是最终落到固定大小的文件块上；程序运行时，又需要把这些文件块读入内存中的页缓冲区，再按整数、字符串或原始字节进行解释。

本章实验只关注 File/ 目录中的最小闭环：BlockID 标识一个文件块，Page 管理内存页内容，FileManager 负责把页写入文件块或从文件块读回页。配套测试已经注册为 FileManagerBasicTest，并通过完整 CTest 验证。

2.2 模块作用

File/ 为上层 buffer/、record/、Log/ 等模块提供块级文件访问能力。它不理解表结构、字段类型或事务语义，只负责三个基础问题：目标块在哪里、页内字节如何读写、内存页如何和本地文件块交换数据。

这种分层让上层模块可以把“读第几个块”“写一页数据”“追加一个新块”作为稳定接口使用，而不必在每个模块里重复处理文件路径、偏移量、fread、fwrite 和块大小计算。

2.3 相关源码文件

表 2-1 列出本章展开的真实源码文件。DBMS_C 当前没有在 File/ 中定义单独的全局 BLOCK_SIZE 常量，块大小由 FileManagerInit(dbDirectoryName, blockSize) 传入，页大小由 PageInit(size) 传入；本章测试使用 TEST_BLOCK_SIZE 400 保持二者一致。

表 2-1 File/Page 相关源码文件

文件	本章关注点
File/BlockID.h	声明文件名和块号组成的块标识结构及访问函数。
File/BlockID.c	实现 BlockIDInit、BlockIDEqual、BlockID2CString 等块标识操作。
File/Page.h	声明页对象以及 int、short、long、string、bytes 的页内读写接口。
File/Page.c	基于 ByteBuffer 实现页内偏移读写和字符串长度前缀编码。
File/FileManager.h	声明文件管理器结构和 read、write、length、append 等块级操作。
File/FileManager.c	负责数据库目录创建、文件句柄缓存、按块偏移读写和追加空块。

从阅读顺序上，建议先看 `BlockID.h/.c`，理解“文件名 + 块号”如何定位块；再看 `Page.h/.c`，理解页内数据布局；最后看 `FileManager.h/.c`，把页和磁盘文件联系起来。

2.4 核心对象职责

图 2-1 展示了三个核心对象之间的关系。它们不是并列工具，而是一条从“定位”到“内存表示”再到“文件读写”的小链路。

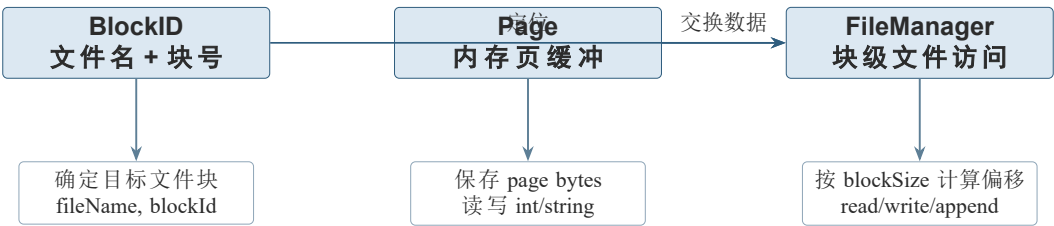


图 2-1 BlockID、Page 与 FileManager 的职责关系

`BlockID` 保存文件名和块号，用于回答“这次读写的是哪个文件的第几个块”。`Page` 是内存中的页级字节缓冲，对上提供 `PageSetInt`、`PageGetInt`、`PageSetString`、`PageGetString` 等接口。`FileManager` 保存数据库目录名、块大小和文件句柄缓存，负责把 `Page` 中的字节写到指定 `BlockID`，或把指定块读回到另一个 `Page`。

2.5 文件块与内存页的关系

数据库底层通常按块或页读写文件。`DBMS_C` 中，文件块是磁盘文件上的一段连续字节，`Page` 是内存中的同样大小的字节缓冲。二者的大小必须在一次读写中保持一致，否则页内偏移和文件块偏移就无法对应。

图 2-2 展示了第 n 个文件块与一个内存 `Page` 的对应关系。实际读写时，`FileManager` 使用 $\text{blockSize} * \text{blockId}$ 计算文件偏移，再用 `fread` 或 `fwrite` 交换整页数据。

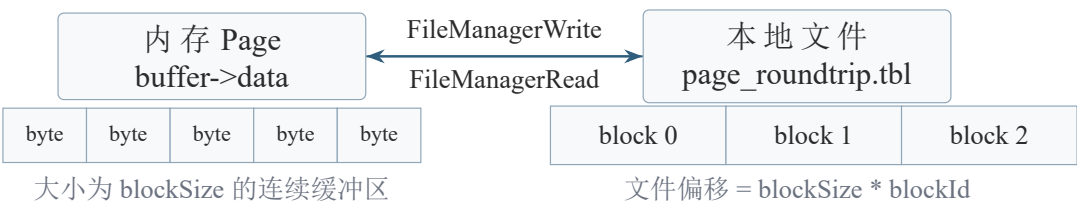


图 2-2 文件块与内存 Page 的对应关系

上层记录和日志最终都依赖这一层完成持久化：记录模块负责解释页内槽位和字段，日志模块负责组织日志记录，而 `FileManager` 只负责把已经组织好的页字节写入对应文件块。

2.6 配套测试

本章新增并注册了配套测试文件：

- `test/File/FileManagerBasicTest.c`

该测试使用 CMocka，测试名为 FileManagerBasicTest。它覆盖以下最小行为：

表 2-2 FileManagerBasicTest 覆盖内容

测试函数	验证目标
test_page_write_read_int	PageSetInt 写入整数后，PageGetInt 能按相同偏移读回。
test_page_write_read_string	PageSetString 写入字符串后，PageGetString 能读回相同内容。
test_block_id_create_compare_and_string	BlockID 能保存文件名和块号，并支持比较和字符串化。
test_file_manager_write_and_read_block	一个 Page 写入块后，另一个 Page 能从同一块读回 int 和 string。
test_file_manager_append_block	FileManagerAppend 能追加块，并让文件长度随块数增长。

测试使用独立测试数据库目录，并在写读块和追加块用例中按进程号生成目录名，避免重复运行 CTest 时被上一次 Windows 文件句柄释放时机影响。测试会尝试在用例前后清理测试目录；如果 Windows 暂时保留目录句柄，后续测试也不会复用同名目录。

2.7 关键代码讲解

代码 2.7 来自 File/BlockID.c。可以看到，BlockID 并不保存文件句柄，只保存文件名副本和块号。比较两个块是否相同，也必须同时比较块号和文件名。

代码 2-1 BlockID 创建与比较

```
1 BlockID *BlockIDInit(CString *name, int id){
2     if (!name)
3         return NULL;
4     BlockID *b = (BlockID *)malloc(sizeof(BlockID));
5     b->fileName = CStringCreateFromCString(name);
6     b->BlockID = id;
7     return b;
8 }
9
10 bool BlockIDEqual(BlockID *b1, BlockID *b2){
11     return BlockIDGetBlockID(b1) == BlockIDGetBlockID(b2)
12     && CStringEqual(BlockIDGetFileName(b1),BlockIDGetFileName(b2));
13 }
```

代码 2.7 来自 File/Page.c。Page 本身把实际字节操作交给 ByteBuffer；字符串写入时先写长度，再写字符内容，读回时按照同样布局恢复 CString。

代码 2-2 Page 中 int 与 string 的读写

```

1  int PageGetInt(Page *p, int position) {
2      int output;
3      bufferGetIntPosition(p->buffer, position, &output);
4      return output;
5  }
6
7  void PageSetInt(Page *p, int position, int data) {
8      bufferPutIntPosition(p->buffer, position, data);
9  }
10
11 CString *PageGetString(Page *p, int position) {
12     p->buffer->position = position;
13     int length = 0;
14     bufferGetInt(p->buffer, &length);
15     if (length < 0) {
16         return NULL;
17     }
18     char *str = (char*)malloc(length + 1);
19     bufferGetBytesPosition(p->buffer, position + sizeof(int32_t), str, length);
20     str[length] = '\0';
21     CString *cString = CStringCreateFromCStr(str);
22     return cString;
23 }
24
25 void PageSetString(Page *p, int position, const CString *cs) {
26     int length = (int)cs->length;
27     p->buffer->position = position;
28     bufferPutInt(p->buffer, length);
29     bufferPutBytes(p->buffer, (uint8_t*)cs->data, length);
30 }

```

代码 2.7 来自 File/FileManager.c。FileManagerRead 和 FileManagerWrite 的核心都是先通过 BlockID 找到文件，再根据块号计算偏移。读不存在的块时，当前实现会把目标 Page 填 0，这让上层可以得到一个空页。

代码 2-3 FileManager 按块读写

```

1  void FileManagerRead(FileManager *fm, BlockID *blockId, Page *page) {
2      FILE *f = FileManagerGetFile(fm, blockId->fileName);
3      if (f == NULL) {
4          perror("File not found");
5          return;
6      }
7      long target_offset = (long)fm->blockSize * BlockIDGetBlockID(blockId);
8      fseek(f, 0, SEEK_END);

```

```

9     long file_size = ftell(f);
10    if (target_offset >= file_size) {
11        memset(page->buffer->data, 0, fm->blockSize);
12        return;
13    }
14
15    fseek(f, fm->blockSize * BlockIDGetBlockID(blockId), SEEK_SET);
16    size_t bytesRead = fread(page->buffer->data, 1, fm->blockSize, f);
17    if (bytesRead < fm->blockSize) {
18        memset(page->buffer->data + bytesRead, 0,
19              fm->blockSize - bytesRead);
20    }
21 }
22
23 void FileManagerWrite(FileManager *fm, BlockID *blockId, Page *page) {
24     FILE *f = FileManagerGetFile(fm, blockId->fileName);
25     if (f == NULL) {
26         perror("File not found");
27         return;
28     }
29     fseek(f, fm->blockSize * BlockIDGetBlockID(blockId), SEEK_SET);
30     size_t bytesRead = fwrite(page->buffer->data, 1, fm->blockSize, f);
31     if (bytesRead != fm->blockSize) {
32         perror("Error! . Failed to read the complete block");
33     }
34     fflush(f);
35 }

```

代码 2.7 展示了追加块的关键逻辑。它先用 `FileManagerLength` 得到新块号，再写入一个全 0 缓冲区，从而扩展文件长度。

代码 2-4 FileManager 追加新块

```

1 int FileManagerLength(FileManager *fm, CString *filename) {
2     FILE *f = FileManagerGetFile(fm, filename);
3     fseek(f, 0, SEEK_END);
4     return (int)(ftell(f) / fm->blockSize);
5 }
6
7 BlockID *FileManagerAppend(FileManager *fm, CString *filename) {
8     int newBlkNum = FileManagerLength(fm, filename);
9     char *buffer = calloc(1, fm->blockSize);
10    BlockID *blk = BlockIDInit(CStringCreateFromCString(filename), newBlkNum);
11    FILE *f = FileManagerGetFile(fm, filename);
12    fseek(f, (long)blk->BlockID * fm->blockSize, SEEK_SET);
13    size_t bytesWritten = fwrite(buffer, 1, fm->blockSize, f);
14    fflush(f);

```

```
15     if (bytesWritten != fm->blockSize) {
16         perror("Failed to append block");
17     }
18     free(buffer);
19
20     return blk;
21 }
```

2.8 运行与观察

本章测试已经接入根目录 CMakeLists.txt。在 D:\textbackslashcode\textbackslashDBMS\textbackslashNewDBMS\textbackslashDBMS_C 下使用固定命令构建和运行：

代码 2-5 构建并运行 File 相关测试

```
1 cmake -S . -B build
2 mingw32-make -C build -j8 SHELL=cmd.exe
3 ctest --test-dir build --output-on-failure -R File
```

如果只想运行本章新增测试，可以使用：

代码 2-6 只运行 FileManagerBasicTest

```
1 ctest --test-dir build --output-on-failure -R FileManagerBasicTest
```

图 2-3 概括了本章最关键的写入再读回过程。测试先在一个 Page 中写入整数和字符串,再调用 FileManagerWrite 写入文件块,随后用另一个 Page 调用 FileManagerRead 读回,并断言内容一致。

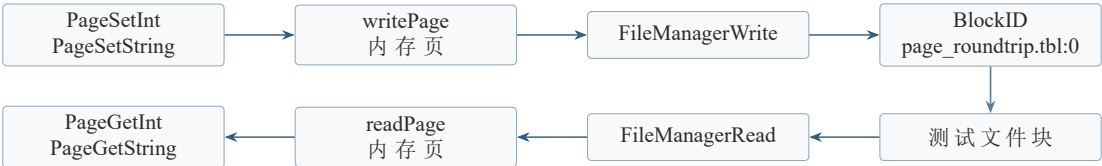


图 2-3 Page 写入、FileManager 落盘、再读回的调用链

运行本章实验时，建议重点观察四件事：第一，Page 写入的数据是否能在同一页中读回；第二，FileManagerWrite 写入块后，另一个 Page 是否能从同一块读回相同内容；第三，BlockID 如何用文件名和块号确定目标位置；第四，这一层如何为后续 BufferManager 提供“读块、写块、追加块”的底座。

2.9 思考题

1. 为什么数据库通常按块或页读写文件，而不是每次只读写一个字段？
2. **Page** 为什么要提供 **int** 和 **string** 的读写接口？这些接口与页内字节布局有什么关系？
3. **BlockID** 为什么需要同时保存文件名和块号？如果只保存块号会有什么问题？
4. 如果一次实验中把块大小从 400 改成更大的值，可能影响哪些上层模块或测试？

第 3 章 日志管理实验

3.1 实验目的

本章用于理解 DBMS_C 如何通过 LogManager 组织日志记录，为后续事务提交、回滚和恢复机制提供基础。日志管理层并不直接解释事务语义，也不执行恢复算法；它的职责是把一条条日志记录追加到日志文件中，在需要时刷盘，并提供从后向前读取日志记录的基础能力。

本章实验只覆盖 Log/LogManager.h 和 Log/LogManager.c 中已经存在的真实接口：初始化日志管理器、追加日志记录、flush 日志页、通过 LogIterator 读取日志记录，以及重新打开后读回已经刷盘的日志内容。

3.2 模块作用

LogManager 位于文件层之上、事务恢复层之下。它依赖 FileManager 完成日志文件的块级读写，依赖 Page 表示内存中的日志页，依赖 BlockID 标识当前日志块。上层事务和恢复模块可以把“写一条日志”“确保某个 LSN 已经刷盘”“从后向前扫描日志”交给这一层完成。

本章不展开 tx/recovery/RecoveryManager.c 和具体日志记录类型。恢复算法会在第 5 章事务、并发与恢复实验中再讨论；本章只把日志追加、日志页组织和日志读取基础讲清楚。

3.3 相关源码文件

表 3-1 列出本章直接或间接涉及的真实源码文件。可以看到，日志管理本身代码量不大，但它建立在第 2 章文件页式存储之上。

表 3-1 LogManager 相关源码文件

文件	本章关注点
Log/LogManager.h	声明 LogManager、LogIterator 以及 append、flush、迭代读取接口。
Log/LogManager.c	实现日志页初始化、日志记录追加、LSN 更新、刷盘和逆序读取。
File/FileManager.h	提供日志文件的块级读写、长度查询和追加新块能力。
File/Page.h	提供日志页内 int 和原始字节读写接口。
File/BlockID.h	标识当前日志文件中的具体块号。
Lib/ByteBuffer.h	LogIteratorNext 返回日志 payload 时使用的字节缓冲对象。
Lib/CString.h	表示日志文件名和测试数据库目录名。

阅读时建议先看 LogManagerInit 如何拿到当前日志块，再看 LogManagerAppend 如何在页内写入记录，最后看 LogIteratorNext 为什么能从后写入的记录开始读取。

3.4 核心对象职责

图 3-1 展示了日志管理层的核心对象关系。LogManager 是中心对象，但它并不直接操作磁盘字节，而是通过 FileManager、Page 和 BlockID 协作完成。

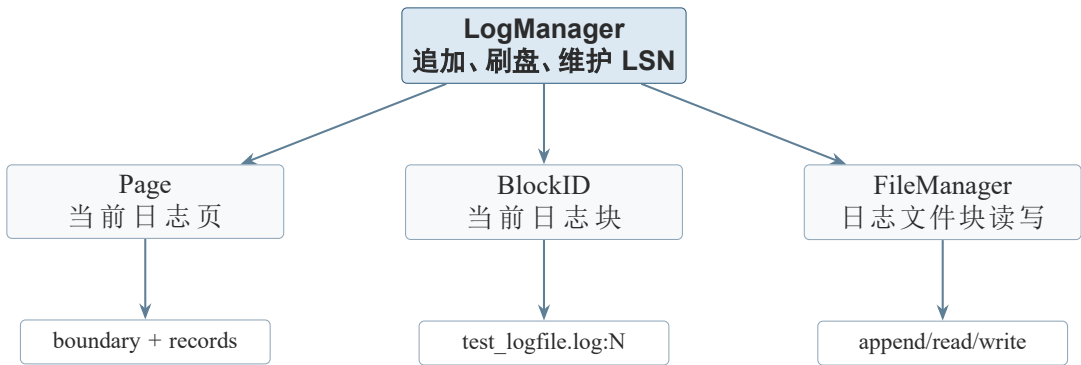


图 3-1 LogManager、FileManager、Page 与 BlockID 的关系

LogManager 维护当前日志文件名、当前日志页、当前日志块、最新 latestLSN 和最近刷盘的 LastSavedLSN。FileManager 负责实际日志文件的块级读写；Page 是日志页在内存中的表示；BlockID 指向日志文件中的一个块。当前实现中存在 LSN：每次 LogManagerAppend 会递增 latestLSN，LogManagerFlush 会把 LastSavedLSN 更新到最新 LSN。

3.5 日志页与日志记录组织

DBMS_C 的日志不是把记录随意散落写入文件，而是通过日志页组织。当前日志页的 0 号偏移处保存一个 boundary，表示本页有效日志记录的起始位置。新日志记录从页尾向前写入：先计算 recordSize，再把位置设为 boundary - recordSize，最后更新页首的 boundary。

图 3-2 描述了这种布局。越新的记录越靠近页前部的 boundary，因此迭代器从 boundary 开始向后移动时，会先读到后写入的记录。

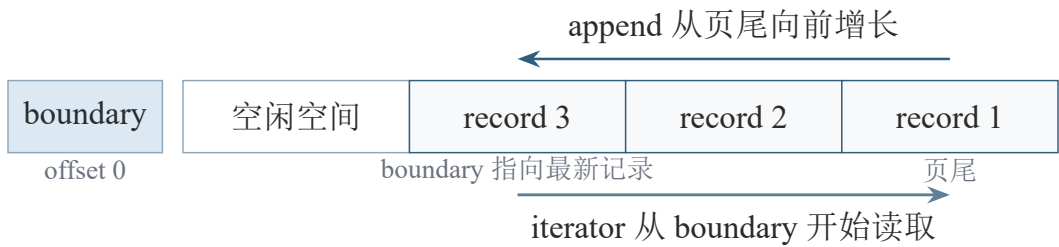


图 3-2 日志页内记录布局示意

每条日志记录由一个内部头部和一段 payload 组成。内部头部在 LogManager.c 中定义为 LogRecordHeader，包含记录总长度和 LSN。payload 的具体含义由上层事务或恢复模块决定，本章测试只把它当作普通字节串。

当当前页空间不足时，LogManagerAppend 会先 flush 当前页，再通过 LogManagerAppendNewBlock 追加一个新的日志块。flush 的作用是把当前日志页

写回日志文件，并记录已经保存到磁盘的 LSN。

3.6 配套测试

本章新增并注册了配套测试文件：

- test/Log/LogManagerBasicTest.c

该测试使用 CMocka，测试名为 LogManagerBasicTest。测试使用独立测试数据库目录和独立日志文件名 test_logfile.log，目录名带进程号和用例计数，避免污染真实数据库文件或重复运行的测试相互影响。

表 3-2 LogManagerBasicTest 覆盖内容

测试函数	验证目标
test_log_manager_append_and_flush	追加一条日志后 LSN 递增，flush 后 LastSavedLSN 更新，日志文件包含一个块。
test_log_manager_iterate_records_reverse_order	连续追加三条文本 payload 后，迭代器按 third、second、first 的顺序读出。
test_log_manager_persists_after_reopen	flush 后销毁并重新打开 FileManager 和 LogManager，仍能读回已经保存的日志。

本章也保留已有 test/Log/LogTest.c，但章节讲解以新增的最小测试为主，因为它只聚焦 LogManager 的基本追加、刷盘和读取能力。

3.7 关键代码讲解

第一段代码来自 Log/LogManager.c 的 LogManagerInit。初始化时，日志管理器会创建一个与文件块大小一致的 Page。如果日志文件还没有块，就追加一个新块；如果已经存在日志块，就读取最后一个日志块作为当前块。

代码 3-1 LogManager 初始化

```
1 LogManager *LogManagerInit(FileManager*fileManager,CString *str){
2     LogManager *logManager = malloc(sizeof (LogManager));
3     logManager->fileManager = fileManager;
4     logManager->logFile = CStringCreateFromCString(str);
5     logManager->logPage = PageInit(logManager->fileManager->blockSize);
6     logManager->LastSavedLSN = 0;
7     logManager->latestLSN = 0;
8     int logSize = FileManagerLength(fileManager,str);
9     if(logSize==0){
10         logManager->currentBlockId = LogManagerAppendNewBlock(logManager);
11     }else{
12         logManager->currentBlockId = BlockIDInit(logManager->logFile,logSize-1);
```

```

13     FileManagerRead(logManager->fileManager, logManager->currentBlockId, logManager->logPage)
14     ;
15     int boundary = PageGetInt(logManager->logPage, 0);
16     if (boundary < logManager->fileManager->blockSize) {
17         LogRecordHeader header;
18         PageGetBytesRaw(logManager->logPage, boundary, (uint8_t*)&header, sizeof(header));
19         logManager->latestLSN = header.lsn;
20     }
21     return logManager;
22 }

```

第二段代码展示了日志追加逻辑。这里的关键点是：日志记录不是从页头顺序写入，而是从页尾向前写入；页首位置 0 保存新的 boundary。

代码 3-2 LogManagerAppend 的核心逻辑

```

1 int LogManagerAppend(LogManager* lm, const uint8_t* payload, uint32_t payloadSize) {
2     uint32_t recordSize = sizeof(LogRecordHeader) + payloadSize;
3     int boundary = PageGetInt(lm->logPage, 0);
4     if (boundary < recordSize + sizeof(int)) {
5         LogManagerFlush(lm);
6         lm->currentBlockId = LogManagerAppendNewBlock(lm);
7         boundary = PageGetInt(lm->logPage, 0);
8     }
9     int pos = boundary - recordSize;
10    LogRecordHeader header = {
11        .length = recordSize,
12        .lsn = ++lm->latestLSN
13    };
14    PageSetBytesRaw(lm->logPage, pos,
15                    (uint8_t*)&header, sizeof(header));
16    PageSetBytesRaw(lm->logPage,
17                    pos + sizeof(header),
18                    payload, payloadSize);
19    PageSetInt(lm->logPage, 0, pos);
20    return header.lsn;
21 }

```

第三段代码展示了 flush。当前实现中，LogManagerFlush 会把当前页写回当前块，并把最近保存 LSN 更新为最新 LSN；LogManagerFlushLSN 则允许上层请求至少刷到某个 LSN。

代码 3-3 日志刷盘

```

1 void LogManagerFlush(LogManager*logManager){
2     FileManagerWrite(logManager->fileManager,logManager->currentBlockId,logManager->logPage);
3     logManager->LastSavedLSN = logManager->latestLSN;
4 }
5
6 void LogManagerFlushLSN(LogManager * logManager,int lsn){
7     if (lsn >= logManager->LastSavedLSN) {
8         FileManagerWrite(logManager->fileManager,logManager->currentBlockId,logManager->logPage
9         );
10        logManager->LastSavedLSN = lsn;
11    }
12 }

```

第四段代码展示了迭代读取。因为 append 把最新记录放在更靠前的位置，迭代器从 boundary 开始向页尾移动时，会自然读出逆序日志。

代码 3-4 LogIterator 读取下一条日志

```

1 ByteBuffer* LogIteratorNext(LogIterator* it) {
2     if (it->currentPos == it->fm->blockSize) {
3         it->blockId->BlockID--;
4         LogIteratorMoveToBlock(it);
5     }
6
7     LogRecordHeader header;
8     PageGetBytesRaw(it->page,
9         it->currentPos,
10        (uint8_t*)&header,
11        sizeof(LogRecordHeader));
12
13    uint32_t payloadSize =
14        header.length - sizeof(LogRecordHeader);
15
16    uint8_t* payload = malloc(payloadSize);
17
18    PageGetBytesRaw(it->page,
19        it->currentPos + sizeof(LogRecordHeader),
20        payload,
21        payloadSize);
22
23    it->currentPos += header.length;
24
25    return bufferInitByteSize(payloadSize, payload);
26 }

```

3.8 运行与观察

在 `D:\textbackslash code\textbackslash DBMS\textbackslash NewDBMS\textbackslash DBMS_C` 下使用固定命令构建和测试：

代码 3-5 构建并运行 Log 相关测试

```
1 cmake -S . -B build
2 mingw32-make -C build -j8 SHELL=cmd.exe
3 ctest --test-dir build --output-on-failure -R Log
```

如果只想运行本章新增测试，可以使用：

代码 3-6 只运行 LogManagerBasicTest

```
1 ctest --test-dir build --output-on-failure -R LogManagerBasicTest
```

图 3-3 概括了追加与刷盘的调用链。上层传入一段 payload，LogManager 加上内部 header 后写入当前日志页；flush 时再把整个页交给 FileManagerWrite 写入日志文件块。

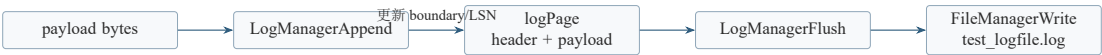


图 3-3 append + flush 调用链

图 3-4 展示了本章测试验证的逆序读取行为。测试依次写入 first、second、third，迭代器读出的顺序是 third、second、first。

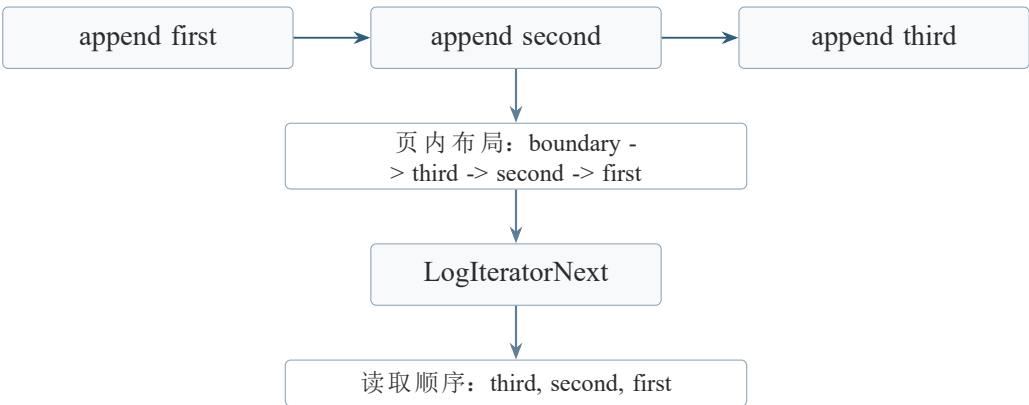


图 3-4 日志逆序读取示意

运行本章实验时，建议重点观察：日志记录是否能被追加；flush 后 LastSavedLSN 是否更新；多条日志记录的读取顺序是否符合当前实现；重新打开后是否能读回已刷盘记录；以及 LogManager 如何依赖第 2 章的 FileManager 和 Page。

3.9 思考题

1. 为什么数据库需要日志，而不是只修改数据页？
2. 为什么日志通常适合顺序追加，而不是频繁在文件中间修改？
3. flush 日志和写回数据页之间可能有什么先后约束？
4. 为什么回滚或恢复过程经常需要从后向前扫描日志？

第 4 章 缓冲区管理实验

4.1 实验目的

本章用于理解 DBMS_C 如何在内存中缓存文件块，减少上层模块直接访问磁盘文件的次数。第 2 章已经看到 `FileManager` 可以把 `Page` 写入文件块或从文件块读回；本章进一步观察 `BufferManager` 如何在文件层之上维护一个固定大小的缓冲池，让上层通过 `pin` 和 `unpin` 使用块。

本章只讨论缓冲区管理的基础行为：缓冲帧、`pin/unpin`、`dirty` 标记、`flush` 写回和替换策略接口。事务回滚、恢复算法和日志记录语义不在本章展开。

4.2 模块作用

`buffer/` 模块位于文件页式存储层之上、记录管理和事务管理层之下。它的核心职责是把磁盘块装入内存 `Buffer`，并在块被修改后按需要写回文件。上层模块不必每次访问记录都直接调用 `FileManagerRead` 或 `FileManagerWrite`，而是先向 `BufferManager` 申请 `pin` 某个 `BlockID`，再通过返回的 `Buffer->page` 读写页内数据。

如果一个 `Buffer` 被 `pin`，表示当前有人正在使用它，不能被替换；如果 `unpin` 后 `pin` 计数归零，它重新变为可用帧。被修改过的 `Buffer` 会通过 `BufferSetModified` 记录事务号和可选 LSN，后续 `flush` 时写回底层文件。

4.3 相关源码文件

表 4-1 列出本章涉及的真实源码文件。这里的 `ReplacementPolicy` 是接口，当前默认实现是 `LRUPolicyCreate` 返回的 LRU 策略。

从阅读顺序看，可以先看 `Buffer.c` 中单个缓冲帧的状态变化，再看 `BufferManager.c` 如何管理多个 `Buffer`，最后看 LRU 策略如何通过统一接口接入。

4.4 核心对象关系

图 4-1 展示了本章对象之间的关系。`BufferManager` 管理一个 `CVector` 形式的缓冲池，池中每个元素是一个 `Buffer*`；每个 `Buffer` 拥有一个 `Page`，并可绑定到某个 `BlockID`。

当前 `BufferManagerInit` 可以接收外部替换策略；如果传入 `NULL`，它会创建默认 LRU 策略。测试 `BufferManagerBasicTest` 使用默认 LRU，但只验证稳定的缓冲池行为，不把 LRU 内部链表细节作为断言目标。

4.5 pin/unpin 机制

`pin` 表示一个块正在被使用。调用 `BufferManagerPin` 时，系统会先查找该块是否已经在缓冲池中；如果已经存在，直接增加该 `Buffer` 的 `pin` 计数；如果不存在，就选择

表 4-1 BufferManager 相关源码文件

文件	本章关注点
buffer/Buffer.h	声明单个缓冲帧，保存 Page、BlockID、pin 计数、事务号和 LSN。
buffer/Buffer.c	实现 BufferPin、BufferUnPin、BufferSetModified、BufferFlush、BufferAssignToBlock。
buffer/BufferManager.h	声明缓冲池、可用帧数量、替换策略和 pin/unpin 等接口。
buffer/BufferManager.c	实现查找已有 Buffer、选择未 pin 帧、pin 块、unpin 帧和按事务刷脏页。
buffer/ReplacementPolicy.h	定义替换策略函数指针接口。
buffer/LRU/LRUCore.h, buffer/LRU/LRUCore.c	实现 LRU 访问记录、victim 查询和移除。
buffer/LRU/LRUPolicy.h, buffer/LRU/LRUPolicy.c	把 LRUCore 包装为 ReplacementPolicy。
File/FileManager.h	Buffer flush 和 assign 时依赖的块级读写接口。
Log/LogManager.h	Buffer flush 时如果存在 LSN，会先调用日志刷盘接口。

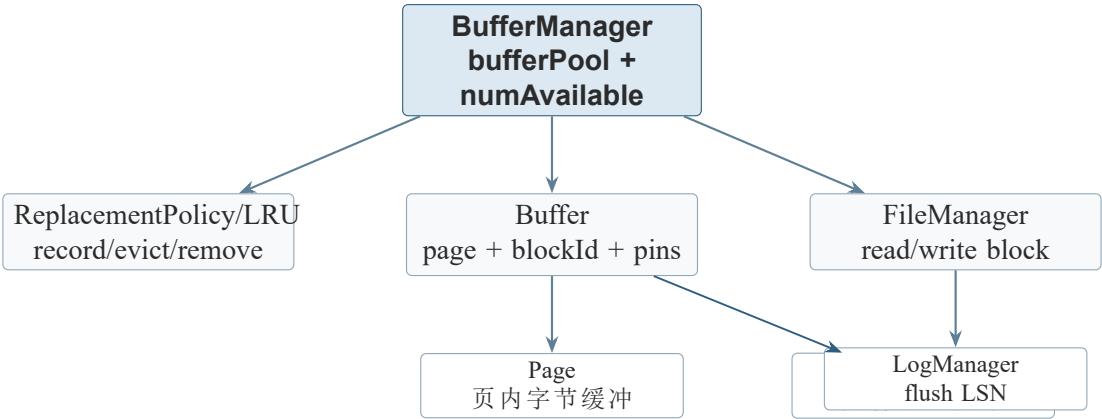


图 4-1 BufferManager、Buffer、Page、BlockID、FileManager 与 LogManager 的关系

一个未 pin 的 Buffer，把它绑定到目标块并读入对应页。调用 `BufferManagerUnpin` 时，pin 计数减少；当 pin 计数降到 0，缓冲帧重新计入可用数量。

图 4-2 展示了单个 Buffer 的状态转换。这里的 dirty 状态不是独立字段，而是由当前实现中的 `txNum >= 0` 表示。

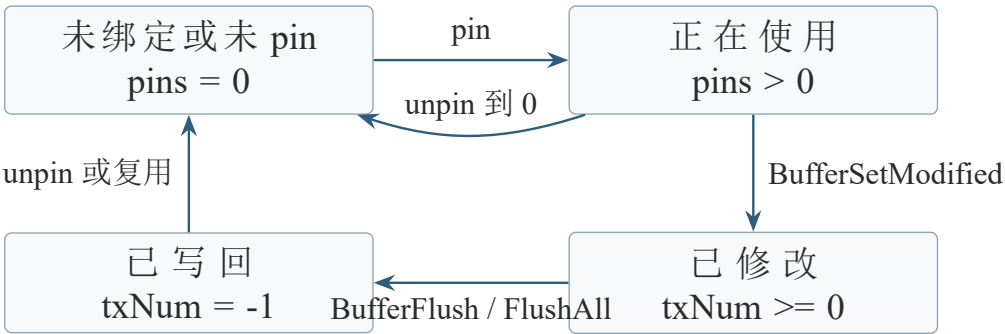


图 4-2 Buffer 状态转换图

图 4-3 展示了 pin/unpin 的调用链。注意，`BufferManagerPin` 会在内部循环调用 `BufferManagerTryToPin`；如果暂时没有可用 Buffer，就等待，超过 `MAX_TIME` 后返回 `NULL`。

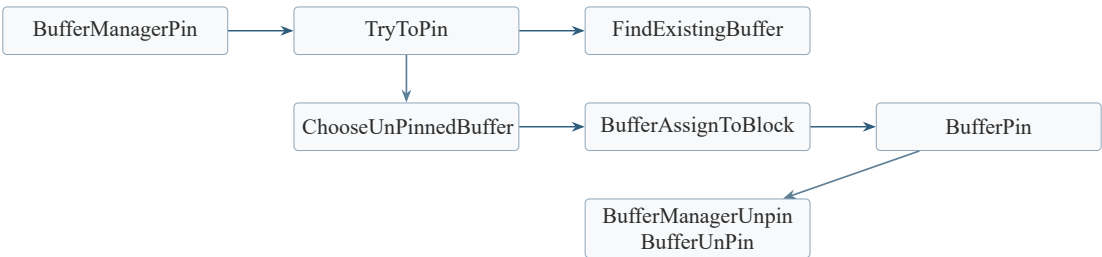


图 4-3 pin/unpin 调用链图

4.6 modified、dirty 与 flush 机制

当前实现中，`BufferSetModified(buffer, txNum, lsn)` 会把 `txNum` 写入 Buffer；如果传入的 `lsn >= 0`，还会记录日志序号。随后 `BufferFlush` 会检查三个条件：Buffer 不为空、已经绑定到某个 `BlockID`、并且 `txNum >= 0`。只有满足这些条件时，Buffer 才会被认为需要写回。

图 4-4 展示了脏页 flush 与文件写回的关系。如果 Buffer 有合法 LSN，flush 会先请求 `LogManagerFlushLSN`；随后再调用 `FileManagerWrite` 把 Page 写回对应文件块。

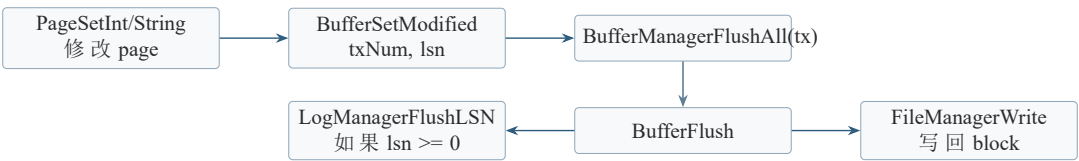


图 4-4 脏页 flush 与文件写回示意图

本章测试使用 `lsn = -1`，只验证 Buffer 修改后通过 `BufferManagerFlushAll` 写回文件；日志先行刷盘的约束会在事务与恢复章节结合真实事务流程再讨论。

4.7 配套测试

本章新增并注册了配套测试文件：

- test/buffer/BufferManagerBasicTest.c

该测试使用独立测试数据库目录，目录名带进程号和用例计数；数据文件为 buffer_basic.tbl,日志文件为 buffer_basic.log。测试使用 FileManager、LogManager 和默认 LRU 策略初始化 BufferManager。

表 4-2 BufferManagerBasicTest 覆盖内容

测试函数	验证目标
test_buffer_manager_pin_and_unpin_updates_available	pin 后可用 Buffer 数减少，unpin 到 0 后恢复。
test_buffer_manager_repin_existing_block	同一 BlockID 重复 pin 返回同一个 Buffer，并正确维护 pin 计数。
test_buffer_manager_dirty_flush_writes_page	修改 Buffer 中的 Page，标记 modified，调用 FlushAll 后能从文件读回。
test_buffer_manager_can_reuse_unpinned_frame	缓冲池满时，unpin 一个帧后可以复用它加载新块。

4.8 关键代码讲解

第一段代码来自 buffer/Buffer.c。它说明单个 Buffer 在绑定新块前会先 flush 自己，随后复制目标 BlockID 并通过 FileManagerRead 读入页面。

代码 4-1 Buffer 绑定到文件块

```
1 void BufferAssignToBlock(Buffer *buffer, BlockID *blockId) {
2     BufferFlush(buffer);
3     buffer->blockId = BlockIDInit(blockId->fileName, blockId->BlockID);
4     FileManagerRead(buffer->fileManager, blockId, buffer->page);
5     buffer->pins = 0;
6 }
```

第二段代码来自 buffer/BufferManager.c 的 BufferManagerTryToPin。这个函数先复用已有缓存；如果块不在池中，就选择未 pin 的 Buffer，并把它分配给目标块。

代码 4-2 BufferManagerTryToPin 的核心逻辑

```

1 Buffer *BufferManagerTryToPin(BufferManager *bm, BlockID *blockId) {
2     Buffer *buffer = BufferManagerFindExistingBuffer(bm, blockId);
3     if (buffer != NULL) {
4         if (buffer->pins == 0)
5             bm->numAvailable--;
6         bm->policy->record_access(bm->policy->impl, buffer->frame_id);
7         BufferPin(buffer);
8         return buffer;
9     }
10
11     buffer = BufferManagerChooseUnPinnedBuffer(bm);
12
13     if (buffer == NULL) {
14         int fid = bm->policy->evict(bm->policy->impl);
15         if (fid < 0)
16             return NULL;
17
18         buffer = *(Buffer**)CVectorAt(bm->bufferPool, fid);
19         if (BufferIsPinned(buffer))
20             return NULL;
21     }
22
23     BufferAssignToBlock(buffer, blockId);
24     bm->numAvailable--;
25     bm->policy->record_access(bm->policy->impl, buffer->frame_id);
26     BufferPin(buffer);
27     return buffer;
28 }

```

第三段代码展示 unpin。只有从 pinned 状态降到 pins == 0 时，numAvailable 才增加；同时替换策略会移除这个 frame 的访问记录。

代码 4-3 BufferManagerUnpin

```

1 void BufferManagerUnpin(BufferManager *bm, Buffer *buffer) {
2     int wasPinned = BufferIsPinned(buffer);
3     BufferUnPin(buffer);
4
5     if (wasPinned && buffer->pins == 0) {
6         bm->numAvailable++;
7         bm->policy->remove(bm->policy->impl, buffer->frame_id);
8     }
9 }

```

第四段代码来自 buffer/Buffer.c 的 BufferFlush。它体现了本章所说的

dirty/flush 规则：只有已绑定块且 `txNum >= 0` 的 Buffer 才会真正写回。

代码 4-4 BufferFlush

```

1 void BufferFlush(Buffer *buffer) {
2     if (buffer == NULL)
3         return;
4
5     if (buffer->blockId == NULL)
6         return;
7     if (buffer->txNum < 0)
8         return;
9
10    if (buffer->lsn >= 0) {
11        LogManagerFlushLSN(buffer->logManager, buffer->lsn);
12    }
13
14    FileManagerWrite(buffer->fileManager, buffer->blockId, buffer->page);
15    buffer->txNum = -1;
16 }

```

4.9 运行与观察

在 `D:\textbackslash code\textbackslash DBMS\textbackslash NewDBMS\textbackslash DBMS_C` 下使用固定命令构建和测试：

代码 4-5 构建并运行 Buffer 相关测试

```

1 cmake -S . -B build
2 mingw32-make -C build -j8 SHELL=cmd.exe
3 ctest --test-dir build --output-on-failure -R Buffer

```

如果只想运行本章新增测试，可以使用：

代码 4-6 只运行 BufferManagerBasicTest

```

1 ctest --test-dir build --output-on-failure -R BufferManagerBasicTest

```

观察测试输出时，可以重点看四件事：第一，`pin` 后 `numAvailable` 是否减少；第二，同一块重复 `pin` 是否仍然返回同一个 Buffer；第三，`unpin` 到 0 后 Buffer 是否重新变为可用；第四，修改页并调用 `BufferManagerFlushAll` 后，底层文件能否读回写入值。

4.10 思考题

1. 为什么数据库需要缓冲池，而不是每次访问记录都直接读写文件？

2. `pin` 计数为什么不能简单设计成布尔值？
3. 当前实现中 `dirty` 状态由 `txNum >= 0` 表示，这种设计有什么优点和局限？
4. `flush` 脏页前为什么可能需要先 `flush` 日志？

第 5 章 事务、并发与恢复实验

5.1 实验目的

TODO: 本章将在补充并通过对应 CTest 后完善。

5.2 相关源码文件

TODO

5.3 配套测试

TODO: 先补充测试，再撰写本章内容。

5.4 实验内容

TODO

5.5 思考题

TODO

第 6 章 记录管理实验

6.1 实验目的

TODO: 本章将在补充并通过对应 CTest 后完善。

6.2 相关源码文件

TODO

6.3 配套测试

TODO: 先补充测试，再撰写本章内容。

6.4 实验内容

TODO

6.5 思考题

TODO

第 7 章 元数据管理实验

7.1 实验目的

TODO: 本章将在补充并通过对应 CTest 后完善。

7.2 相关源码文件

TODO

7.3 配套测试

TODO: 先补充测试，再撰写本章内容。

7.4 实验内容

TODO

7.5 思考题

TODO

第 8 章 SQL 解析实验

8.1 实验目的

TODO: 本章将在补充并通过对应 CTest 后完善。

8.2 相关源码文件

TODO

8.3 配套测试

TODO: 先补充测试，再撰写本章内容。

8.4 实验内容

TODO

8.5 思考题

TODO

第 9 章 查询计划与扫描执行实验

9.1 实验目的

TODO: 本章将在补充并通过对应 CTest 后完善。

9.2 相关源码文件

TODO

9.3 配套测试

TODO: 先补充测试，再撰写本章内容。

9.4 实验内容

TODO

9.5 思考题

TODO

第 10 章 更新执行实验

10.1 实验目的

TODO: 本章将在补充并通过对应 CTest 后完善。

10.2 相关源码文件

TODO

10.3 配套测试

TODO: 先补充测试，再撰写本章内容。

10.4 实验内容

TODO

10.5 思考题

TODO

第 11 章 SQL 执行过程 Trace 可视化实验

11.1 实验目的

TODO: 本章将在补充并通过对应 CTest 后完善。

11.2 相关源码文件

TODO

11.3 配套测试

TODO: 先补充测试，再撰写本章内容。

11.4 实验内容

TODO

11.5 思考题

TODO

第 12 章 综合实验

12.1 实验目的

TODO: 本章将在补充并通过对应 CTest 后完善。

12.2 相关源码文件

TODO

12.3 配套测试

TODO: 先补充测试，再撰写本章内容。

12.4 实验内容

TODO

12.5 思考题

TODO

后续维护原则

后续扩展本指导书时，所有函数名、结构体名、目录路径、测试文件路径和运行结果都必须来自当前仓库的真实状态。若某一章尚未补充并通过对应测试，只能保留 TODO 或说明“后续补充”，不能写入未验证的测试结论。